

CS3423: Compilers I

Practice Session for Lab Evaluation

Department of Computer Science & Engineering, IIT Hyderabad

7 April 2026

Purpose

This document is intended to help you prepare for the upcoming lab evaluation on lexical analysis using LEX/FLEX. You are expected to set up the tools, understand the structure of a lexer, and work through a set of representative exercises.

The language used here is **not** the same as the one that will appear in the exam. The goal is to build familiarity with the toolchain and the underlying concepts.

Learning Objectives

By the end of this session, you should be able to:

- Install and run `flex`.
- Write a `.l` file with appropriate regular definitions and rules.
- Recognise identifiers, numbers, keywords, operators, and comments.
- Apply the longest match rule and resolve rule conflicts.
- Test your lexer against input files systematically.

1 Setting Up LEX/FLEX

1.1 Linux (Ubuntu/Debian)

```
sudo apt update && sudo apt install flex
flex --version
```

1.2 macOS

```
brew install flex
brew link --force flex
```

1.3 Windows

Use Windows Subsystem for Linux (WSL) and follow the Linux instructions.

1.4 Compiling a Lexer

```
flex lexer.l
gcc lex.yy.c -o lexer
```

If you encounter a `yywrap` error, either link with `-lfl` (Linux) / `-ll` (macOS) or define `yywrap()` explicitly.

Run:

```
./lexer < input.txt
```

2 Basic Lexer Structure

A LEX file has three sections separated by `%%`:

```
%{
#include <stdio.h>
}%

DIGIT    [0-9]
LETTER   [a-zA-Z]

%%
{DIGIT}+    { printf("NUMBER: %s\n", yytext); }
{LETTER}+   { printf("ID: %s\n", yytext); }
"|"|"-"|"*"|" "/" { printf("OP: %s\n", yytext); }
[ \t\n]     ;
.           { printf("ERROR: %s\n", yytext); }
%%

int main() { yylex(); return 0; }
int yywrap() { return 1; }
```

3 Practice Example: Tiny C-like Lexer

Create `tiny.l`:

```
%{
#include <stdio.h>
}%

DIGIT    [0-9]
LETTER   [a-zA-Z]
ID       {LETTER}{(LETTER|DIGIT)*}
NUMBER   {DIGIT}+
```

```

WHITE      [ \t\n]

%%
"if"        { printf("KW: if\n"); }
"else"      { printf("KW: else\n"); }
"while"     { printf("KW: while\n"); }

{ID}        { printf("ID: %s\n", yytext); }
{NUMBER}    { printf("NUMBER: %s\n", yytext); }

"+"|"-"|"*"|"|" { printf("OP: %s\n", yytext); }
"="         { printf("ASSIGN: =\n"); }

"("|"")     { printf("PAREN: %s\n", yytext); }
";"         { printf("SEMICOLON: ;\n"); }

">="|"<="|"=="|"!="|">"|"<" { printf("RELOP: %s\n", yytext); }

"/"/".*"    ;
{WHITE}     ;

.           { printf("ERROR: %s\n", yytext); }
%%

int main() { yylex(); return 0; }
int yywrap() { return 1; }

```

3.1 Test Input

```

// comment
x = 10 + 20;
if (x > 0) while (y) y = y - 1;

```

3.2 Expected Output (Complete)

```

ID: x
ASSIGN: =
NUMBER: 10
OP: +
NUMBER: 20
SEMICOLON: ;
KW: if
PAREN: (
ID: x
RELOP: >
NUMBER: 0
PAREN: )
KW: while
PAREN: (

```

```
ID: y
PAREN: )
ID: y
ASSIGN: =
ID: y
OP: -
NUMBER: 1
SEMICOLON: ;
```

3.3 Notes

- Rule order matters. Always place longer patterns first (e.g., `>=` before `>`).
- Keywords must appear before the identifier rule.
- The lexer always follows the **longest match** rule.

4 Testing

```
./lexer < test1.txt > out1.txt
diff out1.txt expected1.txt
```

Automated evaluation will follow a similar approach.

5 Lab Exam Guidelines

- You must write your lexer from scratch during the exam.
- The problem statement will define a different language.
- No pre-written code is allowed.

6 Academic Integrity

1. Use of AI tools (ChatGPT, Copilot, etc.) during the lab session is strictly prohibited. This includes any GenAI apps or CLI (command-line) interfaces to GenAI such as GitHub Copilot in VS Code.
2. Any form of plagiarism or code sharing will result in an **FR Grade**.
3. Submissions will be manually reviewed for similarity and consistency with your understanding.

Final Note

The evaluation is designed to test your understanding of regular expressions and lexical analysis, not memorisation. If you are comfortable writing and debugging small lexers, the exam should be straightforward.